Complemento Práctico: Depuración y Sentencias

Dominando el Debugger de NetBeans y la Sintaxis Estricta

El apunte PDF nos introduce a la teoría de "Expresión vs. Instrucción" y nos muestra cómo poner un *breakpoint*. Sin embargo, en la práctica diaria surgen problemas de gestión: ¿Qué pasa cuando llenamos el código de puntos rojos? ¿Por qué 2 + 2; da error si es una suma válida? Este documento profundiza en la gestión profesional del entorno de depuración, con ejemplos de código diseñados para practicar.

Contenido

Laboratorio de Sintaxis

Diferenciando expresiones de instrucciones con código real.

Gestión de Breakpoints

Administración masiva y puntos condicionales.

Simulación de Debugging

Ejercicios para practicar "Step Over" vs "Step Into".

El Error "Not a Statement"

Java distingue tajantemente entre *calcular algo* (Expresión) y *hacer algo* (Instrucción). Una expresión "suelta" es un desperdicio de procesamiento que el compilador bloquea.

Laboratorio de Errores

Analicemos qué líneas NetBeans marca en rojo y por qué.

```
public class PruebaExpresiones {
 public static void main(String[] args) {
 int a = 10;
 int b = 5;

 // --- CASOS INCORRECTOS (No compilan) ---
 // 1. Cálculo al aire
 // a + b; // Error: "not a statement". Calcula 15 pero no hace nada con él.

 // 2. Comparación suelta
 // a > b; // Error: Calcula "true", pero ese booleano se pierde en el vacío.

 // 3. String suelto
 // "Hola Mundo"; // Error: El texto existe en memoria un milisegundo y se descarta.

 // --- CASOS CORRECTOS (Instrucciones) ---
 // 1. Asignación (Guardamos el resultado)
 int suma = a + b;

 // 2. Uso en método (Enviamos el resultado)
 System.out.println(a > b);

 // 3. Retorno (Devolvemos el resultado)
 // return "Hola Mundo"; (Si estuviéramos en un método que retorna String)
 }
}
```

Diferenciando Expresiones de Instrucciones

Expresión

Calcula un valor

- a + b
- a > b
- "Hola Mundo"

Produce un resultado pero no hace nada con él

Instrucción

Realiza una acción

- int suma = a + b;
- System.out.println(a > b);
- return "Hola Mundo";

Usa el resultado de la expresión para algo concreto

Regla de Oro: En Java, toda expresión debe estar dentro de una instrucción. No puede existir "suelta" en el código porque el compilador la considera un desperdicio de recursos.

Limpieza del Entorno de Trabajo

Gestión de Breakpoints (1/2)

Es común que, tras una sesión intensa de depuración, nuestro proyecto quede lleno de cuadrados rojos olvidados.

El Panel Oculto

No hace falta ir archivo por archivo.

01

Abrir el Panel

Menú: **Window** > **Debugging** > **Breakpoints**

02

Gestionar Breakpoints

Desde aquí pueden seleccionar todos y borrarlos (**Delete All**) o simplemente desactivarlos temporalmente (quedan grises) para no perder la referencia pero que no detengan el código.

Breakpoints Condicionales

Gestión de Breakpoints (2/2)

Imaginemos que tenemos un error que solo ocurre al final de un proceso largo. No podemos presionar F8 mil veces.

Escenario de Práctica

Copien este código en NetBeans. El objetivo es detenerse automáticamente cuando i vale 95.

```
// Bucle largo
for (int i = 0; i < 100; i++) {
    System.out.println("Procesando dato " + i);
    // Supongamos que aquí hay una lógica compleja...
}
```

Pasos

Poner un breakpoint (clic rojo) en la línea del println

Clic derecho sobre el punto rojo > **Breakpoint > Properties**

En "Condition" escribir: `i == 95`

Ejecutar en modo **Debug**

Resultado: El programa imprimirá del 0 al 94 a toda velocidad y se pausará mágicamente en el 95.

Step Over vs Step Into

Navegación en Debugging

Entender la diferencia de estos botones ahorra horas de lectura de código ajeno.

Código para Practicar

```
public class DemoNavegacion {
 public static void main(String[] args) {
 int x = 10;
 int y = 20;
 System.out.println("Iniciando...");
 // [PUNTO A] Pon tu breakpoint aquí abajo
 int total = sumarValores(x, y);
 System.out.println("Total: " + total);
 }

 public static int sumarValores(int a, int b) {
 // [PUNTO B] Código interno
 int resultado = a + b;
 return resultado;
 }
}
```

La Prueba de Navegación

Deténganse en el [PUNTO A].

Step Over (F8)

La línea amarilla saltará directamente al System.out.println("Total..."). El método se ejecutó, pero no vimos cómo.

Cuándo usar: Cuando confías en que el método funciona correctamente y solo quieres ver el resultado.

Step Into (F7)

La línea amarilla "teletransportará" al [PUNTO B], dentro del método sumarValores.

Cuándo usar: Cuando sospechas que el error está dentro de esa función específica.

Regla: Usen **Step Into** solo cuando sospechen que el error está dentro de esa función específica.

Detectando el "Off-by-One"

Debugging de Lógica

El error más clásico en programación es confundir el < con el <=. El debugger es el único que nos muestra esto claramente.

Código Roto (Para arreglar con Debugger)

```
int[] numeros = {10, 20, 30};  // Tamaño 3 (índices 0, 1, 2)

// ERROR COMÚN: Usar <= en lugar de <
for (int i = 0; i <= numeros.length; i++) {
    // Cuando i valga 3, esto explotará.
    // Usen el Watch de Variables para ver cómo 'i' crece hasta romperlo.
    System.out.println(numeros[i]);
}
```

Actividad

Ejecuten esto paso a paso y observen en la ventana "Variables" cómo el valor de i llega a 3. Al intentar ejecutar la línea numeros[3], NetBeans lanzará la excepción ArrayIndexOutOfBounds. ¡Acaban de ver un crash en cámara lenta!

Herramientas de Debugging Dominadas

Breakpoints Básicos

Detener la ejecución en puntos específicos del código para inspeccionar el estado del programa

Breakpoints Condicionales

Pausar la ejecución solo cuando se cumple una condición específica, ideal para bucles largos

Panel de Gestión

Administrar todos los breakpoints desde un único lugar, activarlos, desactivarlos o eliminarlos masivamente

Watch de Variables

Observar en tiempo real cómo cambian los valores de las variables durante la ejecución

Step Over

Ejecutar la línea actual sin entrar en los métodos que llama, ideal para código confiable

Step Into

Entrar dentro de los métodos para inspeccionar su ejecución interna, perfecto para detectar errores

Cierre del Módulo Inicial

Conclusión de la Unidad 1

Con esta actividad cerramos la Unidad 1. Ahora contamos con un arsenal completo:

01

Entorno

JDK y NetBeans configurados

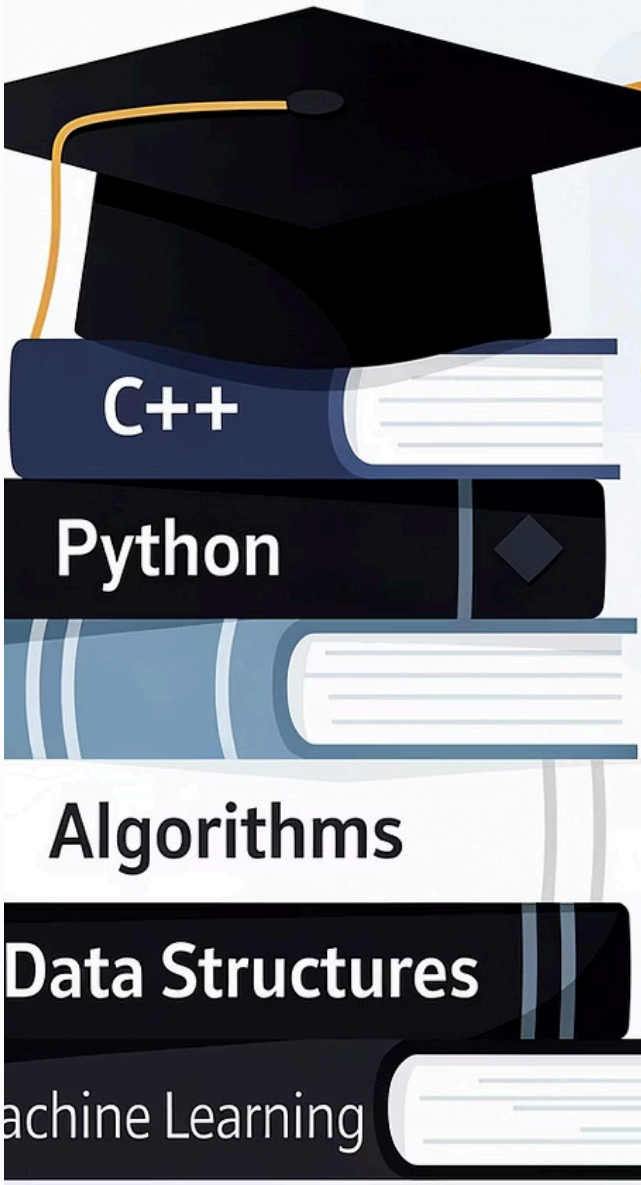
02

Robustez

Scanner validado y manejo de tipos

03

Control

Capacidad de detener el tiempo y ver la memoria (Debugging)

Estamos listos para afrontar la lógica condicional compleja y los bucles en la Unidad 2, sabiendo que si algo falla, tenemos las herramientas para encontrar el error.